

用选择性 Mixup 为 CAT 题目选择去偏 精读笔记

原文: selective-mixup-debiasing-cat_7.pdf · 代码: <https://github.com/xiaoluobouu/MDCAT> (已 clone 到本目录 MDCAT/) · 笔记于 2026-06-27

这篇是 CIKM '25 的工作, 作者来自合肥工业大学。它做的事情一句话概括: 在数据驱动的计算机自适应测试 (CAT) 里, 题目选择模块会被有偏的训练数据带歪——给学得好的人一直出简单题、给学得差的人一直出难题——本文用一种「选择性 Mixup」的数据增强把这个偏差压下去。下面逐块拆开讲。

一、解决什么问题

1.1 先回顾 CAT 在干嘛

CAT (计算机自适应测试) 是个「边考边出题」的系统。它有两个核心部件:

- 认知诊断模块 (CDM): 根据考生已经答过的题, 估计他在各知识点上的能力 θ 。
- 题目选择模块 (Question Selection Module): 根据当前对能力的估计, 决定下一道出什么题。

理想目标是: 用尽量少的题, 把考生的真实能力估准。传统做法靠启发式规则 (比如选「Fisher 信息量最大」的题), 现在主流是数据驱动——直接从海量交互日志里学一个选题网络 (如 BOBCAT、UATS)。

1.2 痛点: 选择偏差 (selection bias)

数据驱动方法虽然准, 但有个被忽视的毛病。训练时用的是双层优化框架: 每个考生的题被分成

- 支撑集 $\mathcal{D}_u$ (80%): 给 CDM 估能力用;
- 元集合 $\mathcal{D}_m$ (20%): 给选题模块当「考卷」, 提供训练反馈。

问题在于: 元集合本身就不平衡。一个学霸的元集合里, 绝大多数是答对的记录; 学渣的元集合里绝大多数是答错的记录。选题模块拼命去对齐这个分布, 结果越训练越极端:

论文图 2a 的观察——高正确率考生的元集合本来集中在 0.6, 训练后漂到 1.0; 低正确率考生漂到 0.0。

于是选题模块学到一个捷径 (shortcut): 「看你是学霸就一直喂简单题, 看你是学渣就一直喂难题」, 根本不管你真实能力。这违背了 CAT 的初衷, 还会让 CDM 的能力估计系统性地高估强者、低估弱者 (图 2b)。

1.3 和已有工作比好在哪儿

之前唯一相关的 UserAIF [16] 是去校准题目侧的参数 (事后补救)。本文不一样: 它直接盯着题目选择模块这个偏差的源头, 在训练阶段就把罕见的「偏差冲突样本」补足。这是 from-the-source 而不是 post-hoc。

二、问题定义 (输入 / 输出)

2.1 符号与维度

- 考生能力向量 $\theta \in (0, 1)^K$, K 是知识概念数 (ASSIST0910 有 119 个概念, NIPS-EDU 有 86 个)。
- 题目 q 的参数 ζ_q : 概念向量 $Q_i \in \{0, 1\}^{1 \times K}$ (这题考哪些知识点)、难度向量 $h_{diff} \in (0, 1)^K$ 、区分度标量 $h_{disc} \in (0, 1)$ 。
- 作答标签 $y \in \{0, 1\}$, 0= 答错, 1= 答对。
- 考生状态 s_t : 长度 = 题目总数的向量, 每个分量取 $\{-1, 0, 1\}$ (答错 / 没做 / 答对)。

2.2 按正确率把考生分三组 (本文的关键设定)

按一个考生在元集合里的正确率, 分成三个属性组:

属性	正确率区间	含义
A	$[0, 0.4]$	以答错为主 (偏「弱」)
B	$(0.4, 0.6]$	答对答错大致平衡 (中等表现, 中性参照)

属性	正确率区间	含义
C	$(0.6, 1.0]$	以答对为主 (偏「强」)

再结合标签 y ，把交互分三类：

- 偏差对齐样本：A 组 + 答错、C 组 + 答对 $\rightarrow$ 多、好学。
- 无偏样本：B 组的所有交互 $\rightarrow$ 中性。
- 偏差冲突样本：A 组 + 答对、C 组 + 答错 $\rightarrow$ 稀少、却最关键（学渣偶尔答对的难题、学霸偶尔答错的题）。

2.3 输入输出小例子

假设知识点 $K = 3$ （代数、几何、概率）。考生 #7 在元集合里答了 10 题，对 2 题、错 8 题，正确率 0.2 $\rightarrow$ 归入 属性 A。

- 输入：他的作答历史 $r = \{(q_1, 0), (q_2, 0), (q_3, 1), \dots\}$ 、题库、CDM 给出的能力估计 $\theta_7^* \in (0, 1)^3$ 。
- 选题模块输出：下一道题 q_t 的编号。
- 问题所在：因为他是 A 组，模块倾向于一直给他出他会答错的难题，于是元集合预测里他几乎全是 0，能力被越压越低。本文要做的，就是把他那 2 道答对的题（偏差冲突样本）的影响放大，别让模块把他一棍子打死。

最终目标：在维持总体准确率的前提下，让最差组（worst group）的预测准确率上来——也就是别让弱势群体被系统性坑掉。

三、模型结构

整体是个两阶段框架，挂在原有双层优化的外层上（CDM 冻结，只训选题模块 ζ_s ）。

阶段 1：跨属性考生检索（Cross-Attribute Examinee Retrieval）

输入：A 组和 C 组里的每个考生。处理：拿他的能力向量 θ^* ，去 B 组里用 L_2 距离找一个最相似的中等表现考生当「中性参照」。输出：每个有偏考生 $\leftrightarrow$ 一个 B 组对应者的配对。

为什么这么做？B 组的人作答平衡、没被偏差带歪，拿他们当锚点，能把 A/C 组的极端样本「拉回中间」。

阶段 2：选择性 Mixup 正则化（Selective Mixup-based Regularization）

输入：阶段 1 的配对（有偏考生 + 其 B 组参照）。处理：在两人标签相同的题目之间做 Mixup（凸组合插值题目参数），合成新的「偏差冲突样本」放进训练。注意这里 Mixup 的是题目参数（难度、区分度、概念向量），考生能力 θ^* 不动。输出：合成集合 $\mathcal{D}_{syn}$ ，连同原始数据一起算损失，更新选题模块。

为什么「标签一致才 Mixup」？因为我们要造的是「难度适中、但标签仍然是答对/答错」的样本。如果把答对和答错混在一起，标签就糊了。只在同标签内插值，等于在「这个标签下，难度从极端到中等」这条线上撒了一堆样本，把决策边界磨平滑。

数据流总结：

冻结的 CDM $\rightarrow$ 内层估出 $*$ $\rightarrow$ 阶段 1：A/C 组每人去 B 组找最近邻
 $\downarrow$
 阶段 2：同标签题目参数做 Beta(,) 插值 $\rightarrow$ 合成样本 $\mathcal{D}_{syn}$
 $\downarrow$
 $L_{final} = L_{emp}(\text{真实}) + \cdot L_{syn}(\text{合成}) \rightarrow$ 梯度更新选题模块 ζ_s

四、关键公式详解

公式 (1)：CDM 的交互函数

$$x = Q_i \circ (\theta - h_{diff}) \times h_{disc}$$

- 符号： $Q_i \in \{0, 1\}^K$ 标记这题考哪些知识点； $\theta \in (0, 1)^K$ 是考生能力； $h_{diff} \in (0, 1)^K$ 是各知识点的难度； h_{disc} 是区分度标量； $\circ$ 是逐元素相乘。

- 举例: $K = 3$, 某题只考第 1、第 3 个知识点 $\rightarrow Q_i = [1, 0, 1]$ 。考生能力 $\theta = [0.8, 0.5, 0.3]$, 题目难度 $h_{diff} = [0.4, 0.4, 0.6]$, 区分度 $h_{disc} = 0.9$ 。
 - $\theta - h_{diff} = [0.4, 0.1, -0.3]$ (正 = 能力高于难度, 应该答对; 负 = 答不出)。
 - 乘 Q_i : $[0.4, 0, -0.3]$ (第 2 维不考, 置 0)。
 - 乘 $h_{disc} = 0.9$: $x = [0.36, 0, -0.27]$ 。
- 作用: 把「能力 vs 难度」的差距编码成特征, 喂给后面的全连接层 (公式 2) 算出答对概率。这一层借鉴了多维 IRT 的思想。

公式 (3): 交叉熵损失

$$\ell(y, p) = -y \log p + (1 - y) \log(1 - p)$$

- 举例: 模型预测答对概率 $p = 0.7$ 。若真实 $y = 1$ (确实答对), $\ell = -\log 0.7 \approx 0.357$ (损失小, 预测对了)。若真实 $y = 0$ (其实答错), $\ell = -\log(1 - 0.7) = -\log 0.3 \approx 1.204$ (损失大, 惩罚)。
- 作用: CDM 和选题模块的基础损失单元, 下面所有损失都是它在不同样本集上的求和。

公式 (5): 内层优化 (估能力)

$$\theta_i^* = \arg \min_{\theta_i} \sum_{t=1}^T \ell(y_{i, q_t}, M(q_t; \theta_i, \zeta_q, \zeta_n))$$

- 含义: 固定题目参数和网络参数, 只调能力 θ_i , 让它对「已选出的 T 道题」的预测最贴近真实作答。 $M(\cdot)$ 就是公式 (1)(2) 那套 CDM 前向。
- 举例: 考生 #7 被选了 5 道题 ($T = 5$), 作答是 $[1, 0, 0, 1, 0]$ 。内层就梯度下降几步 (代码里 $K = 5$ 步) 去找一个 θ_7^* , 使模型在这 5 题上预测的概率尽量接近 $[1, 0, 0, 1, 0]$ 。这就是「诊断」出来的能力。
- 作用: 双层优化的内环, 产出每个考生的能力估计 θ^* , 供外层和检索阶段用。

公式 (6): 外层优化 (训选题模块)

$$\min_{\zeta_s} \frac{1}{N} \sum_{i=1}^N \sum_{q_j \in \mathcal{D}_m} \ell(y_{i, q_j}, M(q_j; \theta_i^*, \zeta_q, \zeta_n))$$

- 含义: 用内层估好的 θ_i^* , 在元集合 $\mathcal{D}_m$ 上算预测损失, 反向只更新新选题网络 ζ_s 。直觉: 如果选题选得好, 估出的 θ^* 就能在没见过的元集合题目上预测准。
- 关键问题: 因为 $\mathcal{D}_m$ 不平衡, 这个 loss 被多数派 (偏差对齐样本) 主导, 于是 ζ_s 越学越偏。本文就是在这个 loss 上动手脚。

公式 (7): 跨属性检索的相似度

$$\text{similarity}_{i,j} = \|\theta_i^* - \theta_j^*\|_2$$

- 含义: 两个考生能力向量的欧氏距离, 越小越相似。
- 举例: A 组考生 #7 能力 $\theta_7^* = [0.30, 0.50, 0.20]$ 。B 组有两个候选: #12 = $[0.35, 0.55, 0.25]$, #20 = $[0.60, 0.40, 0.70]$ 。
 - 距 #12: $\sqrt{0.05^2 + 0.05^2 + 0.05^2} = \sqrt{0.0075} \approx 0.087$ 。
 - 距 #20: $\sqrt{0.30^2 + 0.10^2 + 0.50^2} = \sqrt{0.35} \approx 0.592$ 。
 - $\rightarrow$ 选 #12 当 #7 的中性参照 (能力画像最接近, 但作答更平衡)。
- 作用: 保证 Mixup 是在「语义相近」的人之间做, 造出来的合成样本才合理, 不会把毫不相干的两个人硬混。

公式 (8a-c): 参数级 Mixup

$$Q_{i,j} = \lambda Q_i + (1 - \lambda) Q_j, \quad h_{i,j}^{diff} = \lambda h_i^{diff} + (1 - \lambda) h_j^{diff}, \quad h_{i,j}^{disc} = \lambda h_i^{disc} + (1 - \lambda) h_j^{disc}$$

- 含义: 把有偏考生的题 q_i 和 B 组参照的题 q_j 的题目参数做凸组合, $\lambda \sim \text{Beta}(\alpha, \alpha)$ 。注意混的是题目参数, 不是考生能力。
- 举例: 偏差冲突样本——A 组 #7 答对的一道难题 q_i (难度 $h_i^{diff} = [0.8, 0.8, 0.8]$, 区分度 $h_i^{disc} = 0.9$), 配 B 组 #12 也答对的一道中等题 q_j (难度 $[0.4, 0.4, 0.4]$, 区分度 0.5)。抽到 $\lambda = 0.6$:
 - 合成难度 = $0.6 \times 0.8 + 0.4 \times 0.4 = 0.48 + 0.16 = 0.64$ (介于两者之间, 比纯难题缓和)。
 - 合成区分度 = $0.6 \times 0.9 + 0.4 \times 0.5 = 0.54 + 0.20 = 0.74$ 。
 - 因为 q_i, q_j 都是「答对」(标签=1), 合成题标签仍是 1。
- 作用: 凭空造出一道「难度适中、A 组考生答对」的样本。这种样本在真实数据里几乎没有 (学渣答对难题很罕见), Mixup 把它补出来, 让选题模块知道「A 组的人也能答对中等题」, 从而不再无脑出难题。

公式 (9)(10)(11): 最终去偏目标

$$\mathcal{L}_{syn} = \frac{1}{N} \sum_i \sum_{q_{i,j} \in \mathcal{D}_{syn}} \ell(\dots), \quad \mathcal{L}_{emp} = \frac{1}{N} \sum_i \sum_{q_j \in \mathcal{D}_m} \ell(\dots), \quad \min_{\zeta_s} \mathcal{L}_{final} = \mathcal{L}_{emp} + \omega \mathcal{L}_{syn}$$

- 含义: $\mathcal{L}_{emp}$ 是真实元集合上的经验损失 (= 公式 6), $\mathcal{L}_{syn}$ 是合成偏差冲突样本上的损失, ω 平衡两者。
- 举例: 某 batch 真实损失 $\mathcal{L}_{emp} = 0.62$, 合成样本损失 $\mathcal{L}_{syn} = 0.45$, 取 $\omega = 0.6 \rightarrow \mathcal{L}_{final} = 0.62 + 0.6 \times 0.45 = 0.62 + 0.27 = 0.89$ 。对 ζ_s 求梯度更新。
- 作用: 双目标——既保住真实数据上的预测力 ($\mathcal{L}_{emp}$), 又强制模型重视罕见的偏差冲突情形 ($\omega \mathcal{L}_{syn}$)。 ω 太小没去偏效果, 太大会引入噪声 (论文图 4 的「先升后降」就是这个)。

五、代码实现对照

clone 下来的仓库 MDCAT/ 里, 关键对应关系如下 (已核对源码):

论文内容	代码位置
双层优化主循环、内/外层	train.py 的 inner_algo() (内层 K 步更新 θ)、run_biased() (外层 backward 更新 meta 参数)
选题过程 (公式 4)	train.py:pick_biased_samples() + policy.py 的 StraightThrough 策略
Mixup 去偏总损失 (公式 9-11)	model_cd.py:get_mixup_loss()
跨属性检索 (公式 7)	model_cd.py:get_mixup_loss 内 process_group() 的 torch.norm(stu_emb[indices_1] - temp_stu_emb) + argsort()[1] 取最近邻
参数级 Mixup (公式 8)	model_cd.py:get_mix_output() (NCDM 版在 186-199 行, IRT 版在 141 行附近)

核心对照——get_mixup_loss (model_cd.py:322):

```
indices_0 = where(group_types == 0) # 属性 A
indices_1 = where(group_types == 1) # 属性 B (中性参照池)
indices_2 = where(group_types == 2) # 属性 C

def process_group(group_indices, target_label, ratio, hyp_1):
    for index in group_indices:
        # ——公式 (7): 在 B 组里找 L2 最近邻 ——
        similarities = torch.norm(stu_emb[indices_1] - temp_stu_emb, dim=1)
        closest_indices = indices_1[torch.argsort(similarities)[1]]
        # ——只取 target_label 的题 (标签一致) ——
        label_indices = 该考生中 label==target_label 的题
        label_indices_1 = 最近邻 B 组考生中 label==target_label 的题
        # ——公式 (8): Beta 采样, 对题目参数插值 ——
        lam = np.random.beta(alpha, alpha)
        mix_output = self.cd_model.get_mix_output(temp_stu_emb, exer_pairs, lam)
        mix_loss += compute_mixup_loss(mix_output, labels_pairs) * hyp_1

mix_loss_1 = process_group(indices_0, target_label=1, ...) # A 组 + 答对 → 偏差冲突
mix_loss_2 = process_group(indices_2, target_label=0, ...) # C 组 + 答错 → 偏差冲突
origin_loss += mix_loss_1 + mix_loss_2 # 对应 L_final = L_emp + ·L_syn
```

几个值得注意的实现细节:

1. target_label=1 给 A 组、target_label=0 给 C 组——精确对应论文「偏差冲突样本」定义 (A 组的答对、C 组的答错), 代码只对这两种稀有情形做增强。

2. `get_mix_output (NCDM, 196 行)` : `input_x = mixed_e_disc * (student_embed - mixed_e_diff) * kn_emb` 就是公式 (1) 用混合后的难度 `mixed_e_diff`、区分度 `mixed_e_disc`、概念 `kn_emb`，再过三层 `prednet_full` → 这正是公式 (1)(2) 的前向。考生 `student_embed` 用的是有偏考生本人的 θ^* (`temp_stu_emb`)，印证「只混题目、不混能力」。
3. `ratio` (论文里的 `mix_ratio`) : 控制每个考生采样多少对来混; `hyp_1` 是合成损失的额外权重，配合配置里的 `lam/alpha/mix_ratio` 一起调。
4. 仓库还实现了对比基线: `get_reweight_loss()` (Reweight)、`get_groupDRO_loss()` (GroupDRO)、`cal_env_loss()` (IRM 的多环境)，都在同一框架里，便于公平比较。

预训练好的 CDM 权重在 `history/<dataset>/<cd>-biased/pretrain/.../best_model.pth`，训练入口是 `train.sh / train.py`，CDM 预训练在 `pretrain.py`。

六、小结

实验结论要点 (OOD 评测, 表 2): - 方法主要拉高 **Worst (最差组) 准确率**，相对增益 2.7%~24.64%; Avg. 只小涨 0.1%~1.05%。即「不牺牲整体、专救弱势组」。例: ASSIST0910 + NCDM-BOBCAT@5, Worst 从 0.3662 (ERM) → 0.4989。- IID 下会有轻微掉点 (表 3)，这是去偏方法的通病——拿一点分布内精度换了显著更强的 OOD 泛化。- 消融 (图 5): 三种 Mixup 里 **Mixup-B** (跨属性 A/C↔B) 最稳最好，证明「偏差冲突增强」才是关键，自混 (Self)、同属性混 (Inner) 都不稳。- 超参 (图 4): 对 ω 不敏感，但过大会引入噪声、出现先升后降。- 机制分析 (图 6): 方法把 A 组所选题正确率从趋近 0 拉回 0.4 附近、C 组从趋近 1 拉回 0.6 附近，确实缩小了「所选分布 vs 元集合」的差距。

优点: - 直击偏差源头 (选题模块)，而非事后补救; 即插即用，只需在外层 loss 上加一项，几乎不改 pipeline。- 用「中等表现者 B 组」当中性锚点很巧妙，配上「同标签 Mixup」保证语义不糊。

局限: - IID 精度有轻微 trade-off; ω 需要按 CDM-CAT 组合调 (不同组合行为不同)。- 依赖「按正确率分三组」这个手工设定，阈值 0.4/0.6 是经验值; 属性分组之外的更细粒度偏差没覆盖。- 只去了选题模块的偏差，CDM 侧、题目侧的偏差留给未来工作。

对做 KT/CAT 的启发: 把「样本不平衡 → 模型走捷径」这个视角搬到自适应测试里，并用 Mixup 这种轻量增强解决，是个可复用的套路。如果你在做选题策略 (item selection)，worst-group accuracy 这个评测指标和 OOD (强制元集合 1:1 正负平衡) 的构造方式都值得借鉴。